

Nama : MOHAMMAD RAFLI NUGROHO
NIM : 2303045002

Remedial ETS Kecerdasan Buatan

Soal no 1

CODING

```
import 'package:flutter/material.dart';
import 'package:collection/collection.dart'; // Digunakan untuk PriorityQueue
import 'dart:collection';

void main() {
  runApp(const UhamkaRouteApp());
}
```

```
class UhamkaRouteApp extends StatelessWidget {
  const UhamkaRouteApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'UHAMKA AI Route',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue.shade900),
        useMaterial3: true,
      ),
      home: const HomeScreen(),
    );
  }
}
```

```
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
```

```
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}
```

```
class _HomeScreenState extends State<HomeScreen> {
  bool isKantinMacet = false;
```

```
  // Data Heuristic (Jarak estimasi ke Ruang Dosen)
  final Map<String, int> heuristic = {
    "Gerbang": 8,
    "Lobby": 6,
    "Lab AI": 4,
    "Perpustakaan": 3,
    "Kantin": 2,
    "Parkiran": 5,
    "Ruang Dosen": 0
  };
}
```

```
  // Fungsi untuk membangun graph secara dinamis (berubah jika macet)
  Map<String, List<MapEntry<String, int>>> getGraph() {
    int extraCost = isKantinMacet ? 3 : 0;
    return {
      "Gerbang": [const MapEntry("Lobby", 3), const MapEntry("Parkiran", 5)],
      "Lobby": [const MapEntry("Lab AI", 2), const MapEntry("Perpustakaan", 4)],
      "Lab AI": [const MapEntry("Ruang Dosen", 4)],
      "Perpustakaan": [MapEntry("Kantin", 3 + extraCost)],
      "Kantin": [const MapEntry("Ruang Dosen", 2)],
      "Parkiran": [MapEntry("Kantin", 4 + extraCost)],
      "Ruang Dosen": []
    };
  }
}
```

```
  // --- IMPLEMENTASI ALGORITMA ---
```

```
  // 1. Breadth-First Search (BFS)
  Map<String, dynamic> solveBFS() {
    var graph = getGraph();
    Queue<List<String>> queue = Queue().add(["Gerbang"]);
    Set<String> visited = {"Gerbang"};
```

```
    while (queue.isNotEmpty) {
      var path = queue.removeFirst();
      var node = path.last;
      if (node == "Ruang Dosen") return {"path": path, "cost": _calcCost(path, graph)};
    }
```

```
    for (var edge in graph[node]!) {
      if (!visited.contains(edge.key)) {
        visited.add(edge.key);
```

```

        queue.add([...path, edge.key]);
    }
}
}
return {"path": [], "cost": 0};
}

```

```

// 2. Uniform Cost Search (UCS)
Map<String, dynamic> solveUCS() {
    var graph = getGraph();
    var pq = PriorityQueue<List<dynamic>>((a, b) => a[1].compareTo(b[1]));
    pq.add(["Gerbang", 0]);
    Set<String> visited = {};

```

```

    while (pq.isNotEmpty()) {
        var current = pq.removeFirst();
        List<String> path = List<String>.from(current[0]);
        int cost = current[1];
        String node = path.last;

```

```

        if (node == "Ruang Dosen") return {"path": path, "cost": cost};
        if (visited.contains(node)) continue;
        visited.add(node);

```

```

        for (var edge in graph[node]!) {
            pq.add([...path, edge.key], cost + edge.value);
        }
    }
    return {"path": [], "cost": 0};
}

```

```

// 3. Greedy Best-First Search
Map<String, dynamic> solveGreedy() {
    var graph = getGraph();
    var path = ["Gerbang"];
    while (path.last != "Ruang Dosen") {
        var neighbors = graph[path.last]!;
        if (neighbors.isEmpty) break;
        // Urutkan berdasarkan heuristic saja
        neighbors.sort((a, b) => (heuristic[a.key] ?? 99).compareTo(heuristic[b.key] ?? 99));
        path.add(neighbors.first.key);
    }
    return {"path": path, "cost": _calcCost(path, graph)};
}

```

```

// 4. A* Search
Map<String, dynamic> solveAStar() {
    var graph = getGraph();
    var pq = PriorityQueue<List<dynamic>>((a, b) => a[2].compareTo(b[2]));
    pq.add(["Gerbang", 0, heuristic["Gerbang"]]);
    Map<String, int> minCost = {"Gerbang": 0};

```

```

    while (pq.isNotEmpty()) {
        var current = pq.removeFirst();
        List<String> path = List<String>.from(current[0]);
        int g = current[1];
        String node = path.last;

```

```

        if (node == "Ruang Dosen") return {"path": path, "cost": g};

```

```

        for (var edge in graph[node]!) {
            int newG = g + edge.value;
            if (!minCost.containsKey(edge.key) || newG < minCost[edge.key]!) {
                minCost[edge.key] = newG;
                int f = newG + (heuristic[edge.key] ?? 0);
                pq.add([...path, edge.key], newG, f);
            }
        }
    }
    return {"path": [], "cost": 0};
}

```

```

int _calcCost(List<String> path, Map<String, List<MapEntry<String, int>>> graph) {
    int total = 0;
    for (int i = 0; i < path.length - 1; i++) {
        total += graph[path[i]]!.firstWhere((e) => e.key == path[i+1]).value;
    }
    return total;
}

```

```

@override
Widget build(BuildContext context) {
    final results = [
        {"alg": "BFS", "data": solveBFS()},
        {"alg": "UCS", "data": solveUCS()},
        {"alg": "Greedy", "data": solveGreedy()},
        {"alg": "A*", "data": solveAStar()},
    ];

```

```

    return Scaffold(

```

```

appBar: AppBar(
  title: const Text("UHAMKA Route AI Finder"),
  backgroundColor: Colors.blue.shade900,
  foregroundColor: Colors.white,
),
body: ListView(
  padding: const EdgeInsets.all(16),
  children: [
    _buildConditionCard(),
    const SizedBox(height: 20),
    const Text("Hasil Perbandingan Jalur:", style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
    const SizedBox(height: 10),
    _buildResultTable(results),
    const SizedBox(height: 25),
    _buildConclusion(),
  ],
),
);
}

```

```

Widget _buildConditionCard() {
  return Card(
    elevation: 4,
    child: Padding(
      padding: const EdgeInsets.all(8.0),
      child: Column(
        children: [
          const ListTile(
            leading: Icon(Icons.map_outlined),
            title: Text("Simulasi Kondisi Kampus"),
            subtitle: Text("Ubah variabel lingkungan untuk melihat perubahan rute."),
          ),
          SwitchListTile(
            title: const Text("Kantin Macet (Cost +3)"),
            value: isKantinMacet,
            onChanged: (val) => setState(() => isKantinMacet = val),
            secondary: Icon(Icons.warning, color: isKantinMacet ? Colors.red : Colors.grey),
          ),
        ],
      ),
    ),
  );
}

```

```

Widget _buildResultTable(List<Map<String, dynamic>> results) {
  return SingleChildScrollView(
    scrollDirection: Axis.horizontal,
    child: DataTable(
      headingRowColor: MaterialStateProperty.all(Colors.blue.shade100),
      columns: const [
        DataColumn(label: Text("Algoritma")),
        DataColumn(label: Text("Jalur (Path)")),
        DataColumn(label: Text("Cost")),
      ],
      rows: results.map((r) => DataRow(cells: [
        DataCell(Text(r['alg'], style: const TextStyle(fontWeight: FontWeight.bold))),
        DataCell(Text((r['data']['path'] as List).join(" → "))),
        DataCell(Text(r['data']['cost'].toString())),
      ])).toList(),
    ),
  );
}

```

```

Widget _buildConclusion() {
  return Container(
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: Colors.blue.shade50,
      borderRadius: BorderRadius.circular(12),
      border: Border.all(color: Colors.blue.shade200),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          children: [
            Icon(Icons.stars, color: Colors.blue.shade900),
            const SizedBox(width: 8),
            const Text("Kesimpulan Analisis", style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
          ],
        ),
        const Divider(),
        const Text(
          "Algoritma terbaik adalah **A*** karena ia menggabungkan biaya aktual (UCS) dan informasi heuristic (Greedy).",
          style: TextStyle(fontWeight: FontWeight.w600),
        ),
        const SizedBox(height: 8),
        const Text("• BFS: Hanya mencari jalur terpendek dalam hal jumlah gedung.\n"
          "• UCS: Mencari rute biaya terendah tapi memeriksa terlalu banyak arah.\n"
          "• Greedy: Sangat cepat tapi seringkali tidak akurat karena mengabaikan biaya jalan.\n"
          "• A*: Paling cerdas, efisien, dan selalu menemukan rute terpendek secara biaya."),
      ],
    ),
  );
}

```

UI dan Penjelasan

UHAMKA Route AI Finder

Simulasi Kondisi Kampus
Ubah variabel lingkungan untuk melihat perubahan rute.

Kantin Macet (Cost +3) ☐

Hasil Perbandingan Jalur:

Algoritma	Jalur (Path)	Cost
BFS	Gerbang → Lobby → Lab AI → Ruang Dosen	9
UCS	Gerbang → Lobby → Lab AI → Ruang Dosen	9
Greedy	Gerbang → Parkiran → Kantin → Ruang Dosen	11
A*	Gerbang → Lobby → Lab AI → Ruang Dosen	9

★ Kesimpulan Analisis

Algoritma terbaik adalah **A*** karena ia menggabungkan biaya aktual (UCS) dan informasi heuristic (Greedy).

- BFS: Hanya mencari jalur terpendek dalam hal jumlah gedung.
- UCS: Mencari rute biaya terendah tapi memeriksa terlalu banyak arah.
- Greedy: Sangat cepat tapi seringkali tidak akurat karena mengabaikan biaya jalan.
- A*: Paling cerdas, efisien, dan selalu menemukan rute terpendek secara biaya.

UHAMKA Route AI Finder

Simulasi Kondisi Kampus
Ubah variabel lingkungan untuk melihat perubahan rute.

Kantin Macet (Cost +3) ☒

Hasil Perbandingan Jalur:

Algoritma	Jalur (Path)	Cost
BFS	Gerbang → Lobby → Lab AI → Ruang Dosen	9
UCS	Gerbang → Lobby → Lab AI → Ruang Dosen	9
Greedy	Gerbang → Parkiran → Kantin → Ruang Dosen	14
A*	Gerbang → Lobby → Lab AI → Ruang Dosen	9

★ Kesimpulan Analisis

Algoritma terbaik adalah **A*** karena ia menggabungkan biaya aktual (UCS) dan informasi heuristic (Greedy).

- BFS: Hanya mencari jalur terpendek dalam hal jumlah gedung.
- UCS: Mencari rute biaya terendah tapi memeriksa terlalu banyak arah.
- Greedy: Sangat cepat tapi seringkali tidak akurat karena mengabaikan biaya jalan.
- A*: Paling cerdas, efisien, dan selalu menemukan rute terpendek secara biaya.

Penjelasan Teknis & Analisis

1. Implementasi Algoritma

- BFS: Menjelajahi node per level. Sangat baik untuk mencari jumlah langkah paling sedikit, tapi tidak peduli pada bobot (jarak).
- UCS: Modifikasi BFS yang menggunakan priority queue berdasarkan akumulasi cost. Ia akan selalu menemukan jalur termurah.
- Greedy: Mengandalkan heuristic saja. Cepat, tapi berisiko mengambil jalur sub-optimal jika estimasi meleset.
- A*: Algoritma paling cerdas karena mempertimbangkan biaya tempuh saat ini PLUS sisa jarak ke tujuan.

2. Perbandingan Kondisi (Simulasi Macet)

Berdasarkan kode di atas, jika Anda menekan tombol Switch "Kantin Macet":

- Sebelum Macet: Jalur melalui Kantin (Gerbang → Lobby → Perpustakaan → Kantin → R. Dosen) memiliki cost $3 + 4 + 3 + 2 = 12\$$. Namun jalur Lab AI memiliki cost $3 + 2 + 4 = 9\$$.
- Sesudah Macet: Cost jalur kantin membengkak menjadi 15.
- Hasil: A* dan UCS akan tetap konsisten memilih jalur Lab AI (Cost 9) karena mereka "sadar" biaya.
- Sedangkan Greedy akan tetap memaksa lewat kantin jika nilai heuristic-nya terlihat lebih kecil, meskipun secara total lebih mahal.

3. Perubahan Status Input

Pada bagian Simulasi Kondisi Kampus, switch Kantin Macet (Cost +3) kini berwarna biru (aktif) dengan ikon peringatan merah. Hal ini mensimulasikan adanya hambatan

di jalur yang melewati Kantin (misalnya karena antrean panjang), sehingga biaya perjalanan melalui titik tersebut meningkat sebesar +3.

4. Analisis Hasil Perbandingan Jalur

Tabel menunjukkan bagaimana masing-masing algoritma merespons perubahan biaya tersebut:

a. BFS, UCS, dan A*:

- Ketiga algoritma ini tetap memilih jalur Gerbang -> Lobby -> Lab AI -> Ruang Dosen dengan total Cost 9.
- Mengapa? Karena jalur ini tidak melewati Kantin, sehingga biayanya tetap stabil dan menjadi pilihan paling rasional/termurah dibandingkan harus memutar lewat jalur yang macet.

b. Greedy:

- Jalur yang dipilih adalah Gerbang -> Parkiran -> Kantin -> Ruang Dosen.
- Cost meningkat menjadi 14 (sebelumnya 11 pada gambar pertama).
- Masalahnya: Algoritma Greedy hanya melihat heuristic (jarak estimasi garis lurus) dan menganggap lewat Parkiran/Kantin "terlihat" lebih dekat ke tujuan secara visual, padahal secara biaya nyata jauh lebih mahal karena macet. Ini menunjukkan kelemahan Greedy yang "rakus" dan abai terhadap biaya perjalanan aktual.

Kesimpulan

Algoritma Terbaik: A* (A-Star)

Mengapa A*?

- Optimal & Terjamin: A* menggabungkan kelebihan UCS (akurat dalam biaya) dan Greedy (cepat karena punya arah/heuristic). Dalam kasus ini, ia langsung menemukan jalur lewat Lab AI yang total cost-nya hanya 9.
- Efisiensi Pencarian: Berbeda dengan BFS yang "buta" dan memeriksa semua cabang, A* menggunakan informasi heuristic untuk memprioritaskan arah yang benar-benar menuju Ruang Dosen.
- Adaptif: Ketika ada perubahan beban (seperti Kantin macet), A* secara otomatis akan menghindari jalur tersebut jika ada alternatif lain yang lebih murah secara total.

A* adalah pemenang mutlak untuk sistem navigasi kampus UHAMKA. Ia memberikan efisiensi komputasi yang lebih baik daripada UCS (karena punya "arah" dari heuristic) dan memberikan hasil yang lebih akurat daripada BFS atau Greedy.

Penegasan Kesimpulan

UI bagian bawah menjelaskan mengapa A* tetap menjadi yang terbaik:

- Ketahanan (Robustness): A* berhasil menghindari kemacetan di Kantin karena ia menghitung bobot jalan secara real-time.
- Efisiensi: Dibandingkan Greedy yang terjebak di jalur macet dengan cost 14, A* memberikan rute cerdas dengan cost 9.